LAB 07 AppBar, Drawer, bottom navigation, tabs and themes

Unit 3 • 3-hour guided laboratory • CLO-5

Learning goal. Implement commonly used mobile navigation surfaces and verify their interaction and accessibility.

Core concepts. Scaffold drawer, NavigationBar, IndexedStack, DefaultTabController, TabBar, PopupMenuButton, Material 3 themes.

Before you start. This lab forms a sensible endpoint for the 25-mark midterm practical preparation.

PROJECT / PACKAGE SETUP — run in Terminal

```
flutter create --platforms=android mad_lab07
cd mad_lab07
```

Solved task 1 | Integrated Drawer and bottom navigation

Problem and objective. Navigate between Home, Courses and Account with a Material 3 bottom bar, and open a Drawer destination.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const MaterialApp(home: CampusShell()));

class CampusShell extends StatefulWidget {
  const CampusShell({super.key});
  @override
  State<CampusShell> createState() => _CampusShellState();
}

class _CampusShellState extends State<CampusShell> {
  int page = 0;
  final titles = const ['Home', 'Courses', 'Account'];
  final views = const [
    Center(child: Text('Campus dashboard')),
    Center(child: Text('Enrolled courses')),
    Center(child: Text('Student account')),
  ];

  @override
  Widget build(BuildContext context) => Scaffold(
    appBar: AppBar(title: Text(titles[page])),
    drawer: Drawer(child: SafeArea(child: ListView(children: [
      const UserAccountsDrawerHeader(
        accountName: Text('CSC303 Student'),
        accountEmail: Text('student@example.com')),
      ListTile(leading: const Icon(Icons.home),
        title: const Text('Home'), onTap: () {
          Navigator.pop(context); setState(() => page = 0);
        }),
      ListTile(leading: const Icon(Icons.person),
        title: const Text('Account'), onTap: () {
          Navigator.pop(context); setState(() => page = 2);
        }),
    ]))),
    body: IndexedStack(index: page, children: views),
    bottomNavigationBar: NavigationBar(
      selectedIndex: page,
      onDestinationSelected: (value) => setState(() => page = value),
      destinations: const [
        NavigationDestination(icon: Icon(Icons.home), label: 'Home'),
        NavigationDestination(icon: Icon(Icons.book), label: 'Courses'),
        NavigationDestination(icon: Icon(Icons.person), label: 'Account'),
      ],
    ),
  );
}
```

How the solution works. Scaffold owns the navigation drawer; close the drawer before updating the selected page. NavigationBar modifies an integer selection. IndexedStack retains each child subtree while changing which screen is visible.

Expected output. Bottom navigation and drawer links reach the appropriate label; the app bar title follows the selection.

Check your work. Change pages rapidly and verify bottom selection, drawer behavior and Android Back.

Solved task 2 | Tabbed course workspace with theme and action menu

Problem and objective. Provide Notes and Tasks tabs with a PopupMenuButton that toggles the global theme mode.

lib/main.dart

```
import 'package:flutter/material.dart';

void main() => runApp(const ThemeTabsApp());

class ThemeTabsApp extends StatefulWidget {
  const ThemeTabsApp({super.key});
  @override
  State<ThemeTabsApp> createState() => _ThemeTabsAppState();
}

class _ThemeTabsAppState extends State<ThemeTabsApp> {
  bool dark = false;

  @override
  Widget build(BuildContext context) => MaterialApp(
    theme: ThemeData(colorSchemeSeed: Colors.indigo, useMaterial3: true),
    darkTheme: ThemeData(colorSchemeSeed: Colors.indigo,
      brightness: Brightness.dark, useMaterial3: true),
    themeMode: dark ? ThemeMode.dark : ThemeMode.light,
    home: DefaultTabController(length: 2, child: Scaffold(
      appBar: AppBar(
        title: const Text('Course workspace'),
        actions: [PopupMenuButton<String>(
          onSelected: (v) { if (v == 'theme') setState(() => dark = !dark); },
          itemBuilder: (_) => [PopupMenuItem(
            value: 'theme', child: Text(dark ? 'Light mode' : 'Dark mode'))],
        )],
        bottom: const TabBar(tabs: [
          Tab(icon: Icon(Icons.note), text: 'Notes'),
          Tab(icon: Icon(Icons.task), text: 'Tasks'),
        ]),
      ),
      body: const TabBarView(children: [
        Center(child: Text('Lecture notes appear here')),
        Center(child: Text('Pending lab exercises appear here')),
      ]),
    )),
  );
}
```

How the solution works. DefaultTabController coordinates TabBar and TabBarView; both must have the same number of tabs. ThemeMode switches material palettes at the root so descendant widgets inherit color changes. The popup represents infrequent global actions.

Expected output. Swipe horizontally to switch tabs; use the overflow menu to toggle dark and light appearance.

Check your work. Inspect readability and text contrast in both themes and show that tabs still navigate.

Midterm lab practical preparation

Suggested timed midterm (25 marks): build a two-screen course dashboard with validated search or entry, a responsive list of at least 15 records, working navigation, one dialog and a Git commit. Sample marking: core functionality 12, UI/accessibility 5, architecture/code quality 4, debugging/viva 4. Students should use flutter analyze before submitting.

Scenario-based practice — unsolved

Scenario 1. Three-area faculty app. Choose a correct mix of drawer, tabs and bottom navigation for Home, Timetable, Faculty and Settings. Justify which destinations are primary and which are secondary.

Scenario 2. Theme requirements. Design a night-mode reader with system setting support; explain how you would persist the chosen mode and avoid unreadable combinations.

Technical viva — justify your reasoning

1. Why is IndexedStack useful for bottom navigation state?
2. When is a Drawer a poor substitute for bottom navigation?
3. What invariant must hold between TabBar and TabBarView?
4. How does ThemeData differ from ThemeMode?
5. How can color psychology conflict with minimum accessibility contrast?
6. Why might you use a nested Navigator in a tab?

Submission and self-check

Submit two running navigation examples, themed screenshots and a concise primary/secondary navigation rationale.

CSC303 / NOVEMBER 2026 REVISION

LAB 07 EXTENSION | Midterm practical acceptance specification

The original 25-mark midterm allocation is unchanged. The observable checks below make the grading repeatable without tying students to exactly one sample application.

Criterion	Marks	Observable acceptance
Working features	12	Two screens (4); validated input/search (3); 15+ records (3); working dialog (2)
UI and accessibility	5	Readable portrait/landscape (2); feedback and empty state (2); labels (1)
Code structure	4	Appropriate widget decomposition (2); meaningful naming/state ownership (2)
Debugging and viva	4	Analyze passes (2); demonstrates a failure/fix and explains design (2)

Timed sequence

- Start from an approved starter project and a branch made during the assessment; do not spend the examination period installing Android Studio.
- Implement a small, independently functioning screen before adding secondary UI polish.
- Commit and record the final SHA; run `flutter analyze` and demonstrate on the phone.
- Show one negative input case, one empty or missing record case, and one navigation action with back behavior.

Fairness and contingency

The instructor should provide the same dataset, starter dependencies, phone policy and offline fallback to all candidates. A pre-existing SDK-download failure should be recorded separately from a student implementation defect. Students must still explain code they submit.

Viva probe examples

Which widget owns the filter state? What happens when the result list is empty? Why is a stable key useful when reordering list rows? How would you prevent a route from returning an invalid object?